

Phase 9 - limitations

- [PHASE 9 — LIMITATIONS](#)
 - [1. Decision score component mappings are documented judgement calls](#)
 - [2. Risk completeness is permanently capped at 60/100 by design](#)
 - [3. The Explanation Agent is a deterministic template, not an LLM](#)
 - [4. Decision Policy weights \(§9.4\) are a documented default](#)
 - [5. Two integration bugs were found and fixed during this phase's build](#)
 - [6. Same OBSERVED-data coverage gap as Phases 6/7](#)
 - [7. Model G \(`risk\_score\_v1\_composite` \) is deliberately out of scope for the Prediction Agent and Decision Score](#)
 - [8. No live PostgreSQL was available in this sandbox](#)
 - [9. Full-portfolio performance is not measured](#)
 - [10. Post-delivery audit fix: inherited the Phase 6 ROI bug \(now fixed\)](#)

PHASE 9 — LIMITATIONS

Consolidated from the phase report, per Master Prompt Section 16.

1. Decision score component mappings are documented judgement calls

The linear formulas turning a prediction gap, an IRR spread, or a risk probability into a 0-100 component score (`decision_policy._prediction_score`, `_financial_score`, `_risk_score`) are stated explicitly as *not* fitted to any historical investment-outcome dataset — none exists in this platform. A real deployment with access to genuine historical “which properties turned out well” data should refit these against actual outcomes before relying on the absolute score values, not just their relative ordering.

2. Risk completeness is permanently capped at 60/100 by design

`decision_policy.compute_decision_confidence()` hard-caps the `risk_completeness` component at 60 regardless of what RiskAgent’s status text says, because Phase 7’s Monte Carlo `calibration_status` is permanently `UNCALIBRATED` (§7.7) — no historical panel data exists to calibrate against. This means **no property in this platform can currently reach a decision_confidence much above ~82** (the observed demo-batch maximum), even in the best case. This is intentional and honestly reported, not a defect to “fix” by relaxing the cap — relaxing it would misrepresent how trustworthy the risk signal actually is.

3. The Explanation Agent is a deterministic template, not an LLM

Phase 9’s `explanation_agent.py` produces business-readable text entirely from a Python template reading the structured `DecisionResult` — no LLM is called anywhere in this phase. This keeps the decision-critical path free of any LLM dependency and keeps this phase’s test suite deterministic (`test_e2e_deterministic_reproducibility`). Phase 10’s actual “AI Explanation Layer” (§10.10) is the intended place for LLM-generated prose; it must be built to consume the same `DecisionResult` this template already consumes, and must preserve every numeric value exactly (the same requirement this template already satisfies, tested by `test_e2e_explanation_never_alters_decision_score`).

4. Decision Policy weights (§9.4) are a documented default

Market 20% / Prediction 15% / Financial 25% / Risk 20% / Geo 20% reflects a business-judgement starting point stated plainly in `decision_policy.py`’s module docstring. No historical outcome dataset exists in this

platform to fit these weights against. A real deployment should have the business explicitly review and, if desired, replace them — the same recommendation Phase 8 made for its own Location Score weights.

5. Two integration bugs were found and fixed during this phase's build

- **Module name collisions:** Phases 6/7/8 each named their data-loading module `data_loader.py`. Combined in one process (Phase 9), Python's import cache returned whichever loaded first to every subsequent bare import, regardless of which phase's data was actually needed. Fixed by renaming two of the three copies (`geo_data_loader.py`, `predict_data_loader.py`) — filename-only, zero logic changes, verified by diff.
- **Evidence Agent citation cross-contamination:** joining multiple evidence snippets with `"{content} [{evidence_id}]"` separated by spaces caused Phase 5's sentence-splitter to attach a citation marker to the *next* document's sentence instead of its own, so the Evidence Validator correctly flagged several properties `BLOCKED` (numbers attributed to the wrong source). Fixed by re-terminating each block's punctuation so the citation stays inside its own sentence. A regression test now checks this across 5 properties/localities on every run.

If the underlying evidence corpus or agent output format changes materially, re-run `test_evidence_validator_no_cross_document_contamination` specifically to confirm this class of bug hasn't reappeared in a new form.

6. Same OBSERVED-data coverage gap as Phases 6/7

Properties without both expense and rental history (55.5% of the 8,000-property portfolio, per Phase 6's audit) correctly cascade to `INSUFFICIENT` through the Finance Agent → critical gates → Decision Policy, exactly as intended, rather than receiving a decision built on guessed inputs.

7. Model G (`risk_score_v1_composite`) is deliberately out of scope for the Prediction Agent and Decision Score

Phase 4's registry lists this UNCALIBRATED composite risk model. It is intentionally excluded from Prediction Agent's output and from `decision_policy`'s `risk_score` component, to avoid combining two differently-methodologized, both-uncalibrated risk signals (Model G's composite vs. Phase 7's Monte Carlo) into one number in a way that would be hard to interpret or justify. Phase 10 should decide, with the business, whether Model G's output belongs anywhere in the dashboards as a supplementary, clearly-separate signal.

8. No live PostgreSQL was available in this sandbox

Same situation as every prior phase. All orchestrators default to `--source db` (the production path) but were run here with `--source csv`. Recommend re-running with `--source db` before Phase 10 integration.

9. Full-portfolio performance is not measured

The n=26 demo batch (every agent + 1,000-simulation Monte Carlo per property) completed in well under a minute. No claim is made about runtime at the full 8,000-property scale.

10. Post-delivery audit fix: inherited the Phase 6 ROI bug (now fixed)

`src/finance/engine.py` was pulled into this phase from Phase 6 *before* Phase 6's own post-delivery audit found and fixed a bug in `roi_total_holding_period` (it was computing a gross cash multiple instead of a net return, overstating ROI by exactly 100 percentage points in every case – the same inherited-stale-copy issue independently found in Phase 7). The Decision Policy's `financial_score` uses IRR directly, not this field, so no decision, score, or confidence was ever affected – but

`FinanceAgent.metrics["roi_total_holding_period"]`, surfaced to any downstream consumer reading the full `financial_findings` payload (e.g. Phase 10), was carrying the wrong value. **Fixed** by re-copying Phase 6's corrected `engine.py` (re-verified byte-identical via `diff`), with one new regression test (`test_finance_agent_roi_is_net_return_not_gross_multiple`) added to `test_phase9.py` - suite is now 37/37 passing. The demo-batch decision distribution (INVEST 6 / HOLD 12 / AVOID 7 / INSUFFICIENT 1) was independently re-run and came back byte-for-byte identical, confirming no decision-relevant output was affected.